Absolutely. Since you're learning **Spring Batch**, the easiest way to understand these three is to follow **one CSV record from the file into a Java object**.

Suppose your CSV is:

```
id,firstname,lastname,email,contactno,gender,country,dob
1,Sumit,Chouhan,sumit@example.com,9876543210,Male,India,1995-05-10
```

And you want:

```
    Java

Customer
```

with fields matching those columns.

The overall flow

```
CSV file
   |
   |  "1,Sumit,Chouhan,sumit@example.com,..."
   ▼
LineMapper
   |
   ▼
LineTokenizer
   |
   |  splits the line into fields
   ▼
FieldSet
   |
   ▼
FieldSetMapper
   |
   |  converts fields into Java object
   ▼
Customer object
```

There is an important detail here: `LineMapper` **is the coordinator**, while `LineTokenizer` and `FieldSetMapper` perform the two main pieces of work.

---

1. `LineTokenizer`

What does it do?

`LineTokenizer` takes **one complete line** and breaks it into individual fields.

For example:

```
1,Sumit,Chouhan,sumit@example.com,9876543210,Male,India,1995-05-10
```

becomes conceptually:

```
1
Sumit
Chouhan
sumit@example.com
9876543210
Male
India
1995-05-10
```

It produces a Spring Batch `FieldSet`.

Example

Java

```java
DelimitedLineTokenizer tokenizer = new DelimitedLineTokenizer();

tokenizer.setNames(
        "id",
        "firstname",
        "lastname",
        "email",
        "contactno",
        "gender",
        "country",
        "dob"
);
```

Because this is a CSV file, the delimiter is normally:

```
,
```

So:

```
1,Sumit,Chouhan,sumit@example.com
```

is tokenized into fields.

Simple way to remember

> **LineTokenizer = Split the line.**

2. `FieldSetMapper`

Now we have the individual fields.

For example:

```
id          → 1
firstname   → Sumit
lastname    → Chouhan
email       → sumit@example.com
```

`FieldSetMapper` takes those fields and creates your **Java object**.

For example:

Java

```java
BeanWrapperFieldSetMapper<Customer> mapper =
        new BeanWrapperFieldSetMapper<>();

mapper.setTargetType(Customer.class);
```

Spring Batch can then map:

```
FieldSet
   ↓
Customer
```

So:

```
id = 1
firstname = Sumit
lastname = Chouhan
email = sumit@example.com
```

becomes:

Java

```java
Customer customer = new Customer();

customer.setId(1);
customer.setFirstname("Sumit");
customer.setLastname("Chouhan");
customer.setEmail("sumit@example.com");
```

Simple way to remember

> **FieldSetMapper = Convert fields into Java object.**

---

3. `LineMapper`

Now comes the confusing one.

`LineMapper` is responsible for mapping the **entire line** to your final object.

For example:

```
Java

LineMapper<Customer>
```

means:

```
String line
      ↓
Customer
```

But internally, a typical `DefaultLineMapper` uses:

```
LineMapper
    │
    ├── LineTokenizer
    │       ↓
    │    FieldSet
    │
    └── FieldSetMapper
            ↓
        Customer
```

So `DefaultLineMapper` connects the two.

Conceptually:

```
Java

String line =
    "1,Sumit,Chouhan,sumit@example.com";

FieldSet fieldSet =
    tokenizer.tokenize(line);
```

```
Customer customer =
    fieldSetMapper.mapFieldSet(fieldSet);

return customer;
```

You normally don't have to write this yourself when using `DefaultLineMapper`.

Complete example

For your CSV:

```
id,firstname,lastname,email,contactno,gender,country,dob
1,Sumit,Chouhan,sumit@example.com,9876543210,Male,India,1995-05-10
```

You could configure:

Java

```
@Bean
public LineMapper<Customer> lineMapper() {

    DefaultLineMapper<Customer> lineMapper =
            new DefaultLineMapper<>();

    // 1. Tokenizer
    DelimitedLineTokenizer tokenizer =
            new DelimitedLineTokenizer();

    tokenizer.setNames(
            "id",
            "firstname",
            "lastname",
            "email",
            "contactno",
            "gender",
            "country",
            "dob"
    );

    // 2. FieldSetMapper
    BeanWrapperFieldSetMapper<Customer> fieldSetMapper =
            new BeanWrapperFieldSetMapper<>();

    fieldSetMapper.setTargetType(Customer.class);

    // 3. Connect them
    lineMapper.setLineTokenizer(tokenizer);
    lineMapper.setFieldSetMapper(fieldSetMapper);
```

```
    return lineMapper;
}
```

The important part is:

```java
lineMapper.setLineTokenizer(tokenizer);
lineMapper.setFieldSetMapper(fieldSetMapper);
```

This tells Spring Batch:

```
                DefaultLineMapper
                       |
                       |
        ┌──────────────┴──────────────┐
        ↓                             ↓
  LineTokenizer                FieldSetMapper
        |                             |
        ↓                             ↓
    FieldSet                  Customer object
```

Real example with your CSV

Imagine Spring Batch reads:

```
1,Sumit,Chouhan,sumit@example.com,9876543210,Male,India,1995-05-10
```

Step 1 — `LineTokenizer`

It receives:

```
"1,Sumit,Chouhan,sumit@example.com,9876543210,Male,India,1995-05-10"
```

and creates something like:

```
FieldSet

id        → 1
firstname → Sumit
lastname  → Chouhan
email     → sumit@example.com
contactno → 9876543210
gender    → Male
country   → India
dob       → 1995-05-10
```

Step 2 — `FieldSetMapper`

It receives that `FieldSet` and creates:

```
Java

Customer {
    id = 1,
    firstname = "Sumit",
    lastname = "Chouhan",
    email = "sumit@example.com",
    contactno = "9876543210",
    gender = "Male",
    country = "India",
    dob = ...
}
```

Step 3 — `LineMapper`

`DefaultLineMapper` coordinates the whole process:

```
One CSV line
     ↓
LineTokenizer
     ↓
FieldSet
     ↓
FieldSetMapper
     ↓
Customer
```

The easiest way to remember

Component	Takes	Produces	Main responsibility
LineTokenizer	String/line	`FieldSet`	Break line into fields
FieldSetMapper	`FieldSet`	Java object	Map fields to object
LineMapper	String/line	Java object	Coordinates line → object

In one sentence:

> `LineTokenizer` **splits the line,** `FieldSetMapper` **creates the object, and** `LineMapper` **connects these operations.**

One more important distinction

Don't confuse `LineMapper` with `ItemProcessor`.

In a typical Spring Batch CSV job:

```
CSV File
   ↓
FlatFileItemReader
   ↓
LineMapper
   ↓
Customer
   ↓
ItemProcessor
   ↓
Processed Customer
   ↓
ItemWriter
   ↓
Database
```

For example:

```
CSV
 ↓
"1,Sumit,Chouhan,..."
 ↓
LineMapper
 ↓
Customer
 ↓
ItemProcessor
 ↓
Customer with transformed/validated data
 ↓
ItemWriter
 ↓
MySQL
```

So **LineMapper is part of reading/parsing the input**, while **ItemProcessor is where you apply business transformations/validation**.

If you're following a Spring Batch tutorial using `FlatFileItemReader`, this distinction is particularly important because you'll see all of these classes together.